

AI Agent

Open AI의 기초



python



NumPy



pandas

matplotlib

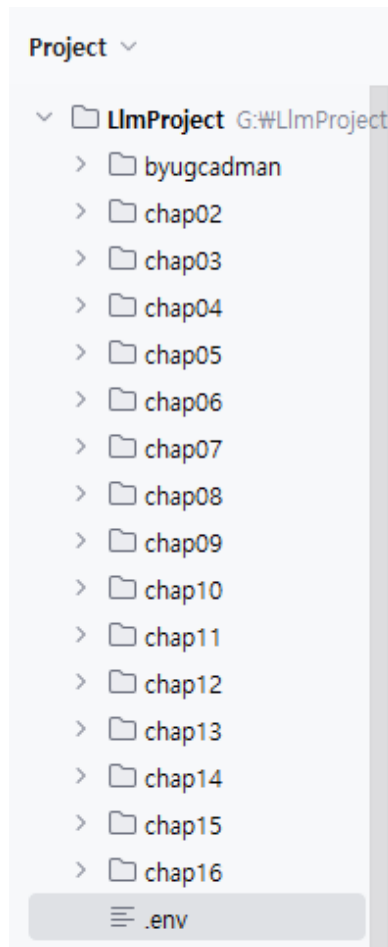
Open AI의 기초

API 키 관리하기

API Key는 타인에게 유출이 되지 않도록 관리해야 합니다.

파일명 : .env

환경 설정 파일



주의 사항

.env 파일은 절대 깃허브(GitHub)나 외부 저장소에 업로드하지 마세요.
사용은 `.env` 파일에 저장 후 `python-dotenv`를 이용해 불러오는 방식을 권장합니다.
API Key가 노출되면 과금 위험이 있습니다.

≡ Open AI의 기초

≡ API 키 관리하기

API Key는 타인에게 유출이 되지 않도록 관리해야 합니다.

openai-apikey.py

요구 사항

.env 파일 내용에서 특정 Key 정보를 읽어서 환경 변수로 등록합니다.
OpenAI API Key를 일부는 그대로 출력하고, 나머지는 *로 가리는(masking) 역할을 합니다.

출력 결과

```
.env 파일의 위치 : G:\python\llm\myenv\.env

API Key를 일부만 출력
TEST_API_KEY : [hello *****]

# 하나의 Key 정보를 확인합니다.
TEST_API_KEY : [hello world]

# 여러 개의 Key 정보를 확인합니다.
환경 변수 'MADONNA_KEY'를 찾을 수 없습니다.
AMADEUS_CLIENT_ID : asdf
MADONNA_KEY : None
TEST_API_KEY : hello world
```

≡ Open AI의 기초

≡ 질문하고 답변 받기

OpenAI의 API 키를 사용하여 질문하고 그에 대한 답변을 받는 프로그램을 작성해 보세요. `question_and_answer.py`, `question_and_answer_result.txt`

요구 사항

GPT에게 2022년 월드컵 우승팀을 물어 보고 그에 대한 답변을 받아 오는 예시입니다.

출력 결과

response

```
ChatCompletion(id='chatcmpl-D0LLO9MAz6sSsybRG3IMVyV95icbQ', choices=[Choice(finish_reason='stop', index=0, logprobs=None, message=ChatCompletionMessage(content='2022년 FIFA 월드컵에서는 아르헨티나가 우승을 차지했습니다. 아르헨티나는 결승전에서 프랑스를 상대로 승리하여 월드컵 트로피를 들어올렸습니다.', refusal=None, role='assistant', audio=None, function_call=None, tool_calls=None, annotations=[]))], created=1768974418, model='gpt-4o-2024-08-06', object='chat.completion', service_tier='default', system_fingerprint='fp_deacdd5f6f', usage=CompletionUsage(completion_tokens=52, prompt_tokens=30, total_tokens=82, completion_tokens_details=CompletionTokensDetails(accepted_prediction_tokens=0, audio_tokens=0, reasoning_tokens=0, rejected_prediction_tokens=0), prompt_tokens_details=PromptTokensDetails(audio_tokens=0, cached_tokens=0)))
```

`response.choices[0].message.content`

2022년 FIFA 월드컵에서는 아르헨티나가 우승을 차지했습니다. 아르헨티나는 결승전에서 프랑스를 상대로 승리하여 월드컵 트로피를 들어올렸습니다.

≡ Open AI의 기초

≡ 질문하고 답변 받기

OpenAI의 API 키를 사용하여 질문하고 그에 대한 답변을 받는 프로그램을 작성해 보세요.

question_and_answer.py

부연 설명

응답의 usage 항목을 살펴 봅니다.

이 예시의 요청은 입력 30 토큰 + 출력 52토큰 = 총 82토큰을 사용했습니다.

OpenAI API 요금은 이 총 토큰 수 기준으로 부과 됩니다.

항목	설명
입력 토큰	prompt_tokens=30 system 메시지 "You are a helpful assistant." user 메시지 "2022년 월드컵 우승팀은 어디야?"
출력 토큰	completion_tokens=52 모델이 생성해준 문장 전체가 여기에 해당합니다. 2022년 FIFA 월드컵에서는 아르헨티나가 우승을 차지했습니다. 아르헨티나는 결승전에서 프랑스를 상대로 승리하여 월드컵 트로피를 들어올렸습니다.
전체 토큰	total_tokens = 82
총 비용	(입력 30 토큰 + 출력 52 토큰) × 모델 단가

≡ Open AI의 기초

≡ temperature 매개 변수

temperature 매개 변수의 값을 변경해 가면서, 얼마나 창의적으로 답변하는 지를 확인하도록 합니다. temperature.py, temperature_text.txt

권장 예시

추천 예제 1: 창의적인 이야기 생성

"content": "고양이가 운영하는 커피숍에 대한 짧은 이야기를 만들어 주세요."

temperature=0 → 비교적 일관된 이야기

temperature=2 → 매우 다양한 전개

추천 예제 2: 상품 이름 만들기

"content": "AI를 활용한 스마트 우산의 브랜드 이름을 5개 만들어 주세요."

temperature가 높을수록 독특한 이름 생성

추천 예제 3: 여행 슬로건 만들기

"content": "부산을 홍보하는 여행 슬로건을 10개 만들어 주세요."

추천 예제 4: 가상의 국가 설정

"content": "**아틀란티스라는 가상의 나라의 수도와 국기를 상상해서 설명해 주세요.**"

☰ Open AI의 기초

☰ 스트리밍 방식과 비스트리밍 방식

스트리밍 방식과 비스트리밍 방식을 비교해 봅니다.

non-streaming.py, streaming.py, streaming.py.result.txt

요구 사항

비스트리밍(Non-Streaming) 방식은 모델이 전체 응답 생성을 완료할 때까지 클라이언트는 대기하는 방식입니다.

스트리밍(Streaming) 방식은 모델이 텍스트를 생성하는 즉시 서버는 응답을 작은 청크(Chunk) 단위로 쪼개어 클라이언트에 지속적으로 전송하는 방식입니다.

≡ Open AI의 기초

≡ Responses api 방식

OpenAI의 최신 버전인 Responses API에 대하여 학습해 보도록 합니다.

responses-basic.py, responses-basic_result.txt

질문 사항

2022년 월드컵 우승팀이 어느 나라인지 한 문장으로 요약해줘.

질문 사항

```
response
Response(id='resp_057b3b73b4f2acf0006a1a6a2f2ba4819a99ac43d0a69b51a6', created_at=1780116015.0, error=None, incomplete_details=None,
instructions='You are a helpful assistant.', metadata={}, model='gpt-5-nano-2025-08-07', object='response',
output=[ResponseReasoningItem(id='rs_057b3b73b4f2acf0006a1a6a304eb0819a9afcf5d241a9a368', summary=[], type='reasoning', content=None,
encrypted_content=None, status=None), ResponseOutputMessage(id='msg_057b3b73b4f2acf0006a1a6a35da9c819a96243fc6b5dc5516',
content=[ResponseOutputText(annotations=[],
text='2022년 월드컵의 우승팀은 아르헨티나이다.', type='output_text', logprobs=[])], role='assistant', status='completed', type='message',
phase=None)], parallel_tool_calls=True, temperature=1.0, tool_choice='auto', tools=[], top_p=1.0, background=False,
completed_at=1780116022.0, conversation=None, max_output_tokens=None, ... 이하 중략
```

output_text

2022년 월드컵의 우승팀은 아르헨티나이다.

≡ Open AI의 기초

≡ Responses streaming 방식

OpenAI의 최신 버전인 Responses API에 대하여 streaming 방식을 사용해 보도록 합니다.

responses-streaming.py

질문 사항

홍길동전을 한 문장으로 요약해줘.

특이 사항

리다이렉션을 사용하여 텍스트 파일의 결과를 확인해 주세요.

```
python responses-streaming.py > responses-streaming.py.result.txt
```

AI Agent

프롬프트 엔지니어링



python



NumPy



pandas

matplotlib

프롬프트 엔지니어링

2가지 서로 다른 상황에 대하여 동일한 질문에 대한 프롬프트 엔지니어링을 구현해 보겠습니다.

Prompt Engineering

Implementing prompt engineering for two distinct scenarios with the same question



Common Question:
I'm feeling tired today, what
drink would be good?

2가지 상황

상황 01 : 바리스타

상황 02 : 한의사

공통 질문

오늘 좀 피곤한데 뭐 마시면 좋을까?

프롬프트 엔지니어링

2가지 서로 다른 상황에 대하여 동일한 질문에 대한 프롬프트 엔지니어링을 구현해 보겠습니다.

Prompt Engineering Result

Barista's Response

Feeling tired today?
For days like this, I recommend an **'Americano'** with a strong **espresso** shot, or a sweet and creamy **'Caramel Macchiato'**. These will perk you up with a good dose of caffeine. If you want something especially cozy, a **'Vanilla Latte'** would also be nice. What sounds good to you?



Acupuncturist's Response

If you're feeling tired, I recommend some herbal teas that can boost your energy and soothe your body. **'Jujube Tea'** is great for replenishing energy and aiding fatigue recovery. **'Chrysanthemum Tea'** helps relieve eye fatigue and clear your mind. Lastly, warm **'Ginger Tea'** warms the body and aids digestion to help relieve fatigue. Take a moment to rest and nurture your body with care.



프롬프트 엔지니어링

프롬프트 엔지니어링을 사용하여 질문하고 그에 대한 답변을 받는 프로그램을 작성해 보세요.

role_prompt_engineering.py

질문 내용

```
messages=[
  {
    "role": "system",
    "content": role_info["content"]
  },
  {
    "role": "user",
    "content": user_question
  }
]
```

[barista_at_cafe.py](#), [korean_oriental_medicine.py](#) 소스를 병합한 예시입니다.

출력 결과

[역할: 바리스타]

오늘 피곤하신가요? 그럼 활력을 줄 수 있는 카페인이 가득한 에스프레소 샷이나, 부드러우면서도 각성 효과가 있는 카페라떼를 추천해드리고 싶어요. 만약 우유가 포함된 음료를 선호하지 않으신다면, 아메리카노도 좋은 선택이 될 수 있습니다. 모두 여러분의 에너지를 북돋아줄 거예요! 어떤 걸로 드릴까요?

오늘 피곤함을 느끼신다면 몸에 무리가 가지 않도록 가볍고 따뜻한 차를 드시는 것이 좋습니다. 대추차나 생강차를 권해드립니다. 대추차는 몸을 따뜻하게 하고 피로 회복에 도움을 주며, 생강차는 기운을 북돋아 주고 소화를 돕는 데 효과적입니다. 마실 때에는 천천히 한 모금씩 음미하며 드시면 좋습니다. 충분한 수면과 휴식도 함께 취하셔야 합니다. 몸이 보내는 신호에 귀 기울여 주십시오.

프롬프트 엔지니어링

프롬프트 엔지니어링을 사용하여 질문하고 그에 대한 답변을 받는 프로그램을 작성해 보세요.

barista_at_cafe.py

질문 내용

```
messages=[
  {
    "role": "system",
    "content": "너는 커피 전문점의 바리스타야. 항상 친절하고 메뉴를 추천해주는 말투로 답변해."
  },
  {
    "role": "user",
    "content": "오늘 좀 피곤한데 뭐 마시면 좋을까?"
  }
]
```

출력 결과

```
response.choices[0].message.content
```

오늘 피곤하신가요? 그런 날에는 **에스프레소** 샷이 들어간 카페인 가득한 '아메리카노'나 부드럽고 달콤한 '카라멜 마키아토'를 추천드릴게요. 에너지를 충전하시는데 도움이 될 거예요. 아니면 특별히 따뜻한 느낌이 필요하시다면 '바닐라 라떼'도 좋습니다. 어떤 게 끌리시나요?

프롬프트 엔지니어링

프롬프트 엔지니어링을 사용하여 질문하고 그에 대한 답변을 받는 프로그램을 작성해 보세요.

korean_oriental_medicine.py

질문 내용

```
messages=[
  {
    "role": "system",
    "content": "너는 한의학에 정통한 전통 한의사다. 몸의 기운과 컨디션을 고려하여 차분하고 조언하듯 답변해라."
  },
  {
    "role": "user",
    "content": "오늘 좀 피곤한데 뭐 마시면 좋을까?"
  }
]
```

출력 결과

response.choices[0].message.content

피곤함을 느끼신다면 기운을 북돋우고 몸을 편안하게 해줄 수 있는 차를 추천드립니다. **대추차**는 기운을 보강하고 피로 회복에 도움을 줄 수 있습니다. 또한 **국화차**는 눈의 피로를 풀어주고 머리를 맑게 해주는 효과가 있어 좋습니다. 마지막으로 따뜻한 **생강차**는 몸을 따뜻하게 하고 소화기능을 촉진시켜 피로를 덜어주는 데 도움이 됩니다. 차를 마시면서 잠시 휴식을 취하며 정성을 다해 심신을 돌봐주세요.

퓨샷 프롬프팅

퓨샷 프롬프팅을 사용하여 질문하고 그에 대한 답변을 받는 프로그램을 작성해 보세요.

`few_shot_exam.py`, `few_shot_exam.py.result.txt`

질문 내용

```
python -X utf8 few_shot_exam.py > few_shot_exam.py.result.txt
```

```
messages=[
  {
    "role": "system",
    "content": "너는 사전(Dictionary)이야. 동일한 형식으로, 간단하고 객관적으로 정의만 제시해줘."
  },
  {"role": "user", "content": "연필"},
  {"role": "assistant", "content": "연필: 글을 쓰거나 그림을 그리는 데 사용하는 도구."},
  {"role": "user", "content": "컵"},
  {"role": "assistant", "content": "컵: 물이나 음료를 담아 마시는 데 사용하는 그릇."},
  {"role": "user", "content": "의자"},
  {"role": "assistant", "content": "의자: 사람이 앉을 수 있도록 만든 가구."},
  {"role": "user", "content": "모자"}
]
```

출력 결과

```
response.choices[0].message.content
```

모자: 머리에 쓰는 의류 액세서리로, 보호나 장식의 목적으로 사용됨.

제로샷과 퓨샷 프롬프팅

제로샷과 퓨샷 프롬프팅을 사용하여 '무지개'에 대한 질문을 하고 그에 대한 답변을 받는 프로그램을 작성해 보세요.

rainbow.py

출력 결과

zero-shot

무지개는 햇빛이 비나 이슬 속 물방울에서 굴절되고, 색들이 다르게 굴절해 분산된 빛이 물방울 안에서 반사된 뒤 다시 밖으로 나와 관찰자의 눈으로 들어올 때 생기는 광학 현상입니다.

주요 내용

- 형성 원리: 굴절 → 분산 → 물방울 내부 반사 → 굴절
- 색 순서(외곽에서 안쪽으로): 빨강, 주황, 노랑, 초록, 파랑, 남색, 보라
- 주된 무지개는 한 물방울에서 다층의 빛이 모여 생기고, 보조 무지개는 물방울에서 두 번의 내부 반사를 거쳐 생깁니다. 보조 무지개의 색 순서는 주된 무지개와 반대로 보일 수 있습니다.
- 관찰 조건: 태양이 등에 있고 앞쪽에 비가 내리거나 안개가 있을 때, 빛의 입사각이 약 42도에서 가장 잘 보입니다.

더 궁금한 점이 있나요? 예를 들어 무지개를 직접 관찰하는 방법이나 실험 아이디어를 원하시면 알려 주세요.

few-shot

A: red, orange, yellow, green, blue, indigo, violet

싱글턴과 멀티턴

싱글턴과 멀티턴 프로그램을 구현해 보겠습니다.

turn_example.py, turn_example.txt, turn_question.txt

질문 내용

```
messages=[
    {"role": "system", "content": "너는 사용자의 학습을 돕는 친절한 튜터야. 답변은 2~3문장 이내로 간결하게 해."},
    {"role": "user", "content": user_input},
],
)
```

출력 결과

사용자: 안녕~ 나는 홍길동이라고 해.

AI: 안녕하세요, 홍길동님! 만나서 반갑습니다. 오늘 어떻게 도와드릴까요?

사용자: 내 누군지 아니?

AI: **미안하지만, 난 네가 누군지 알 수 없어.** 그러나 네가 필요로 하는 정보를 찾거나 질문에 답변하는 데 도움을 줄 수 있어! 무엇을 도와줄까?

사용자: exit

싱글턴과 멀티턴

싱글턴과 멀티턴 프로그램을 구현해 보겠습니다.

turn_example.py, turn_example.txt, turn_question.txt

출력 결과

사용자: 파이썬에서 변수가 뭐야?

AI: 파이썬에서 변수는 데이터를 저장할 수 있는 메모리 공간을 의미합니다.

사용자: 변수 이름은 아무거나 써도 돼?

AI: 변수 이름은 몇 가지 규칙과 권장 사항을 따르는 게 좋아요. 파이썬에서 변수 이름을 정할 때의 기본적인 규칙은 다음과 같습니다:
규칙을 지키면서 의미 있고 명확한 변수 이름을 선택하면 코드의 유지보수성과 가독성을 높일 수 있습니다.

사용자: 변수 이름 예시 하나만 보여줘

AI: 물론이죠! 일반적인 변수 이름의 예시를 하나 보여드리겠습니다.

```
total_price = 100
```

사용자: exit

streamlit 실행 테스트

streamlit을 사용하여 다음과 같이 간단한 프로그램을 작성해 보세요.

streamlit_widget.py

구현 절차

1. `app.py` 파일을 생성하고 다음 내용을 입력합니다.

```
import streamlit as st
```

```
st.title("Hello Streamlit~~")
```

```
st.write("Streamlit 앱이 정상적으로 실행되고 있습니다.")
```

2. 터미널에서 실행합니다. 이메일을 물어 보면 대략적으로 입력합니다.

```
streamlit run app.py
```

3. 브라우저가 자동으로 열리며 `localhost:8501`에서 앱이 실행되는지 확인합니다.

4. 코드 수정 후 저장하면 브라우저가 자동으로 새로 고침되는지 확인합니다.

streamlit 위젯 실습

streamlit의 위젯 정보를 사용하여 다음과 같이 간단한 프로그램을 작성해 보세요.

streamlit_widget.py

실행 결과



Hello Streamlit

Streamlit 앱이 정상적으로 실행되고 있습니다.



입력 위젯 데모

이름

나이

 - +

주 사용 언어

 v

확인

입력이 완료되었습니다.

입력 결과

이름 :

나이 : 25

언어 : Python

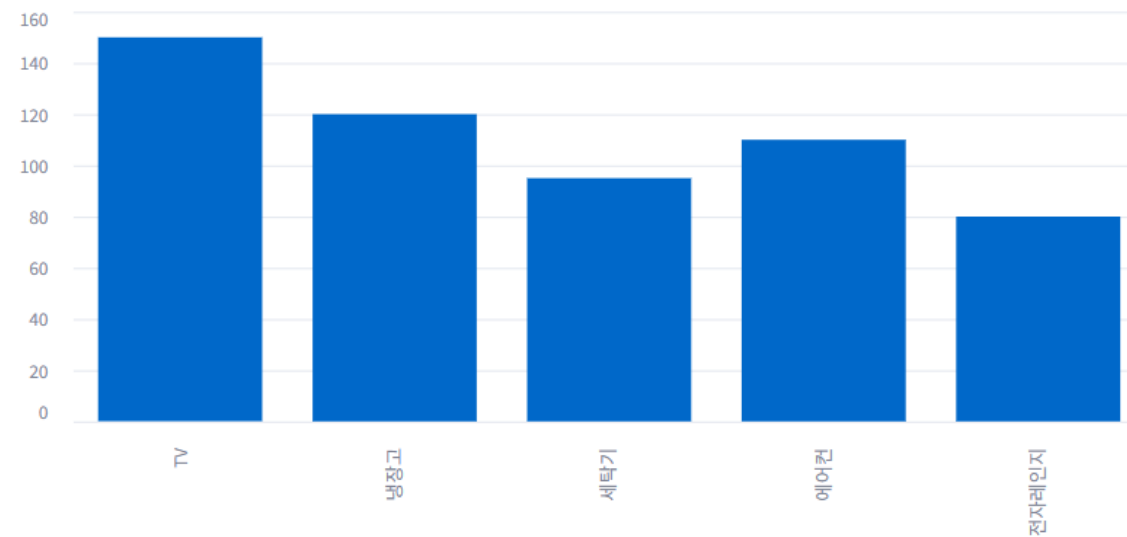
출력 요소

streamlit의 출력 요소를 사용하여 다음과 같이 간단한 프로그램을 작성해 보세요.

streamlit_output_components.py

가전 제품 판매량 예제

가전제품	판매량
0 냉장고	120
1 세탁기	95
2 TV	150
3 전자레인지	80
4 에어컨	110



streamlit으로 chatbot 만들기

다음 요구 사항과 같이 구현해주는 프로그램을 작성해 보세요.

streamlit_exam.py

요구 사항

streamlit으로 chatbot을 구현해 봅니다.
system 메시지로 말의 길이를 제한합니다.
max_tokens 매개 변수로 출력 상한을 둡니다.
최근 대화만 기억하게 하여 토큰 비용을 최소화하도록 합니다.

잘못된 API 키



S 너는 사용자의 학습을 돕는 친절한 튜터야. 답변은 2~3문장 이내로 간결하게 해줘.

무엇을 도와드릴까요?

계속 진행하려면 귀하의 OpenAI API key가 필요합니다.

Your message



streamlit으로 chatbot 만들기

다음 요구 사항과 같이 구현해주는 프로그램을 작성해 보세요.

streamlit_exam.py

출력 결과

Chatbot

S 너는 사용자의 학습을 돕는 친절한 튜터야. 답변은 2~3문장 이내로 간결하게 해줘.

 무엇을 도와드릴까요?

 안녕~ 나는 홍길동이라고 해.

 안녕하세요, 홍길동님! 만나서 반가워요. 어떤 도움을 드릴까요?

 내 누군지 아니?

 홍길동은 한국의 고전 소설 "홍길동전"의 주인공으로, 불의에 맞서 싸우고 정의를 실현하는 인물로 잘 알려져 있어요. 혹시 그 홍길동을 말씀하시는 건가요?

Your message



AI Agent

GPT 비전 사용하기

 python

 NumPy

 pandas

 matplotlib

≡ GPT 비전 사용하기

≡ 이미지 사이즈 변경

비전을 사용하여 이미지 작업을 수행시 큰 이미지는 분석을 못할 수 있습니다.

image_resize.py

요구 사항

이번 예시는 GPT 비전과 직접적인 연관은 없습니다.

하지만, 로컬 파일 실습시 이미지 사이즈가 너무 크면 다음과 같은 메시지가 나올수 있습니다.

```
'type': 'invalid_request_error', 'param': None, 'code': 'invalid_image_url'}}
```

이미지 크기가 너무 크거나, 너무 큰 해상도는 처리를 잘 하지 못합니다.

원본 이미지에 대하여 resize 작업을 수행하도록 하겠습니다.

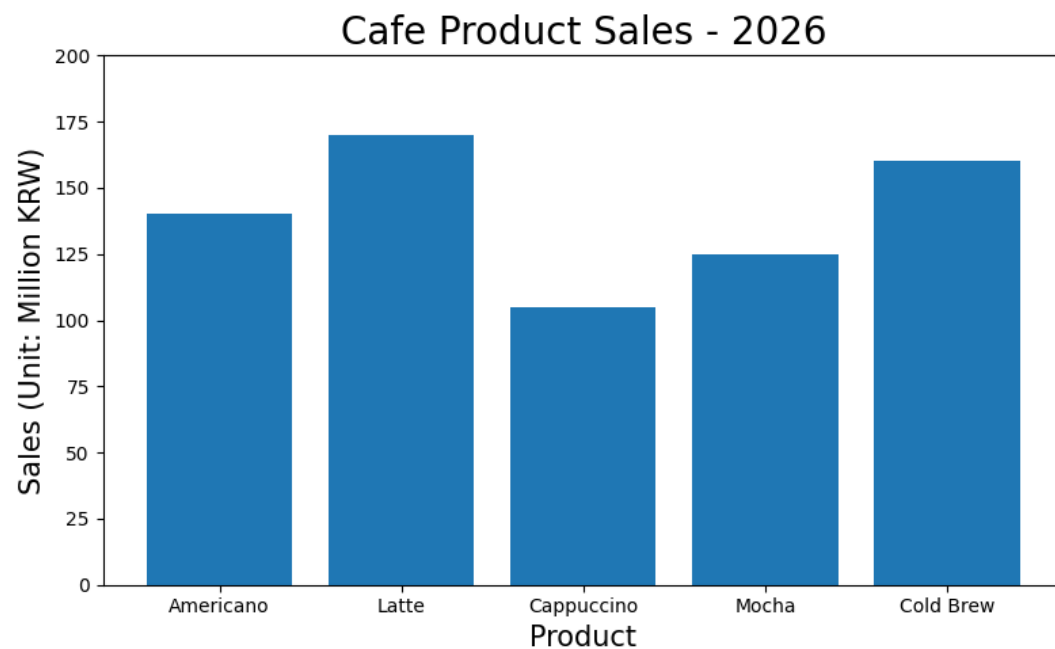
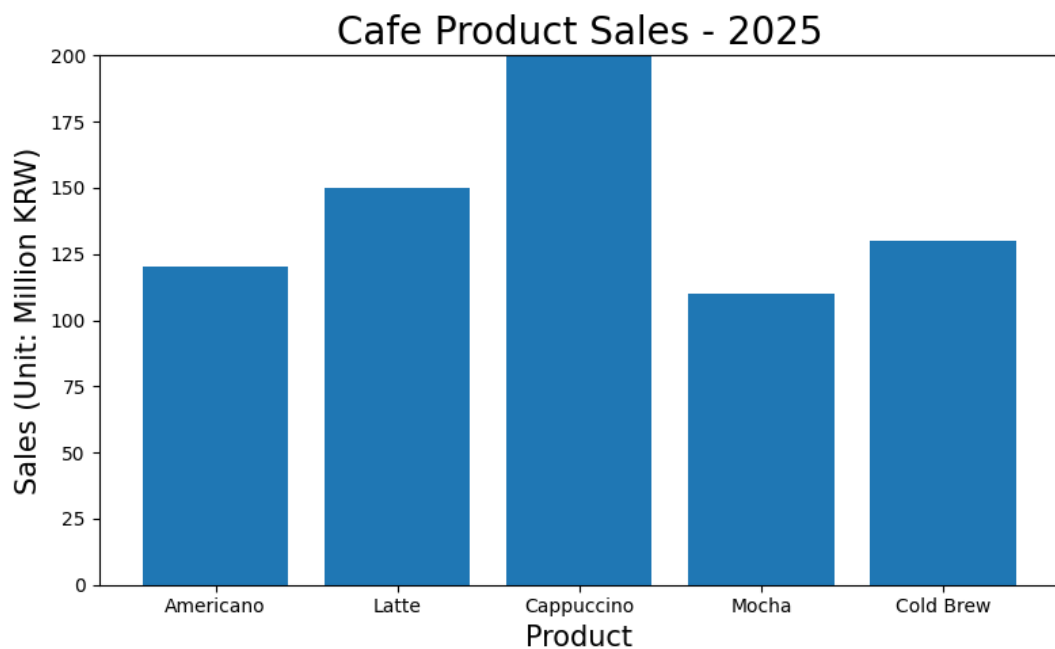
☰ GPT 비전 사용하기

☰ 비교 이미지 생성하기

비전을 사용하여 유사한 이미지에 대한 분석 작업을 수행하기 위하여 Chart 데이터를 만드는 예시입니다.

cafe_sales.py

차트 데이터 생성



≡ GPT 비전 사용하기

≡ 인터넷 이미지 설명 요청

특정 이미지에 대한 설명 정보를 요청하는 프로그램을 작성해 보세요.

image_explanation.ipynb

요구 사항



출력 결과

한 여성이 주방에서 계란을 휘젓고 있습니다. 그녀는 검은색 운동복을 입고 있으며, 옆에는 프라이팬이 있습니다. 밝고 즐거운 분위기입니다.

≡ GPT 비전 사용하기

≡ 로컬 이미지 설명 요청

특정 이미지에 대한 설명 정보를 요청하는 프로그램을 작성해 보세요.

image_explanation.ipynb

요구 사항



출력 결과

맑은 날씨에 바다와 바위들이 있는 해안 풍경입니다. 멀리 작은 배가 보입니다.

≡ GPT 비전 사용하기

≡ 이미지 비교해보기

특정 이미지에 대한 설명 정보를 요청하는 프로그램을 작성해 보세요.

image_explanation.ipynb

요구 사항



출력 결과

첫 번째 이미지는 숲 속 산책로에서 찍은 것으로, 나무가 많고 사람들이 걷고 있습니다. 두 번째 이미지는 화려한 꽃과 전통 건축물이 있는 정원에서 찍은 것으로, 배경...

출력 토큰을 제한했기 때문에 중간에 출력 결과가 잘려서 보입니다.

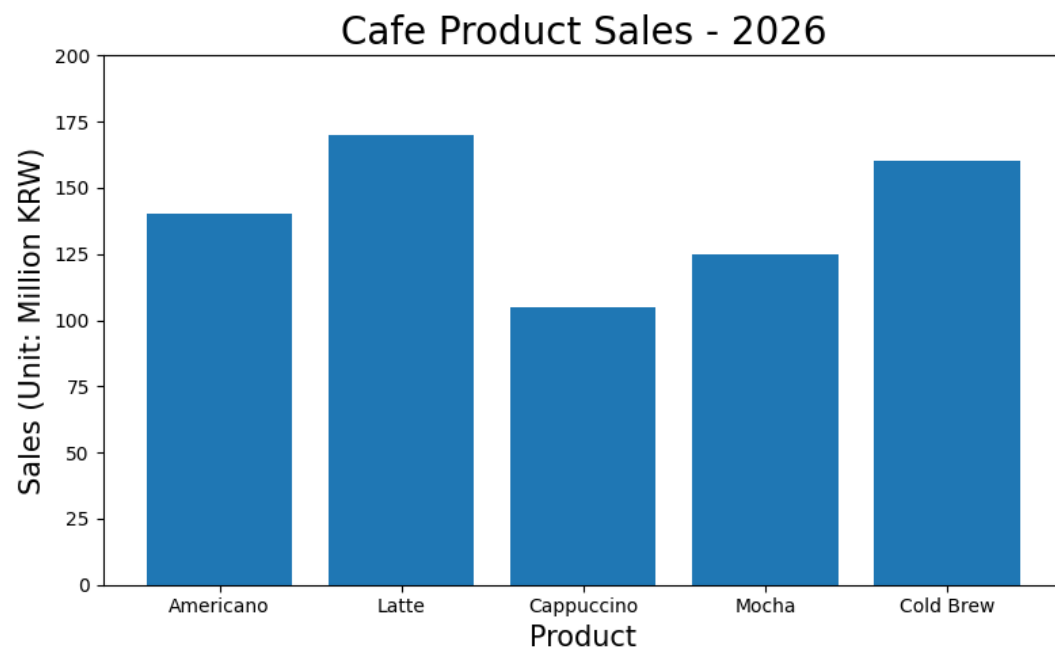
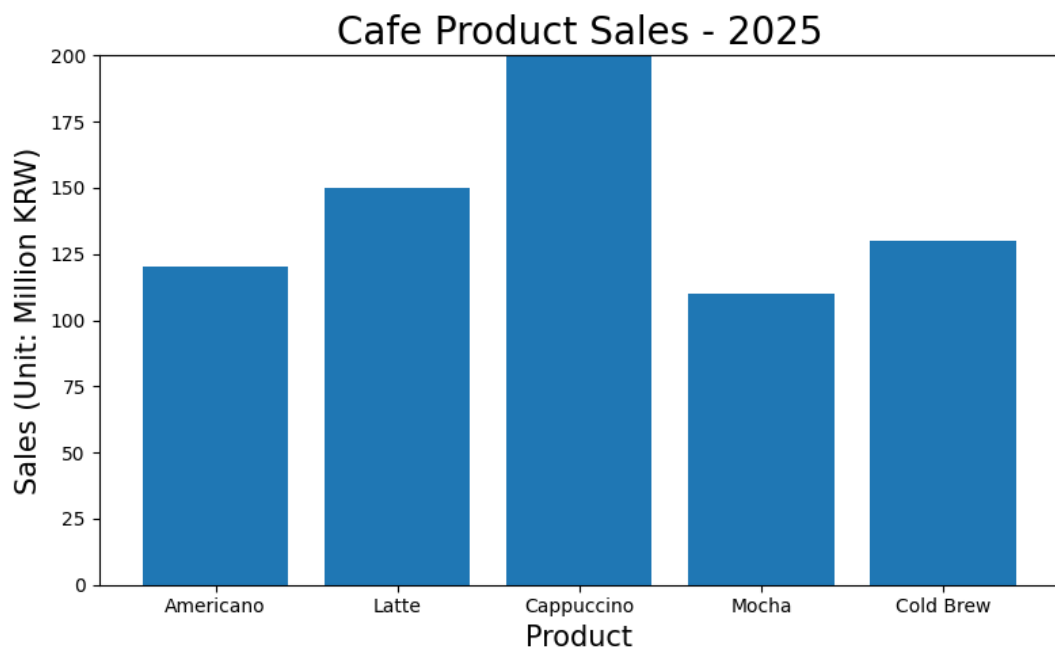
☰ GPT 비전 사용하기

☰ 그래프 이미지 비교해보기

특정 이미지에 대한 설명 정보를 요청하는 프로그램을 작성해 보세요.

image_explanation.ipynb

요구 사항



출력 결과

2025년에서 2026년으로 넘어가면서 카푸치노 판매가 감소하고, 라떼와 콜드 브루 판매가 증가했습니다. 모카는 약간 증가했습니다.

≡ GPT 비전 사용하기

≡ 이미지를 사용하여 퀴즈 만들기

특정 이미지들에 대한 4지선다형 객관식 퀴즈를 만드는 프로그램을 작성해 보세요.

image_quiz_generator.py

결과 예시

문제 1

![image](...)

Q: 이미지 설명 중 틀린 것은?

- (1) 바다가 보이는 풍경이다.
- (2) 사진 한가운데에 등대가 있다.
- (3) 꽃이 있는 화단이 보인다.
- (4) 태양광 패널이 설치되어 있다.

정답: (2)

이유: 가운데 있는 건물은 등대처럼 보이는 구조물이지만, 실제 등대는 아니다.

문제 1



Q: 이미지 설명 중 틀린 것은? (1) 바다가 보이는 풍경이다. (2) 사진 한가운데에 등대가 있다. (3) 꽃이 있는 화단이 보인다. (4) 태양광 패널이 설치되어 있다.

정답: (2)

이유: 가운데 있는 건물은 등대처럼 보이는 구조물이지만, 실제 등대는 아니다.

AI Agent

Function Calling



python



NumPy



pandas

matplotlib

Function Calling

사칙 연산

Function Calling을 사용하여 사칙 연산을 수행하는 코드를 작성해 보세요.

math_main_console.py, math_functions.py

요구 사항

다음 코드는 사칙 연산을 하기 위하여 Function Calling에 전달하기 위한 **tools** 코드 내용입니다.

```
"name": "calculate",
"description": "두 숫자에 대해 사칙연산을 수행합니다.",
"parameters": {
  "type": "object",
  "properties": {
    "operation": {
      "type": "string",
      "description": "연산 종류(add, sub, mul, div)"
    },
    "a": {
      "type": "number",
      "description": "첫 번째 숫자"
    },
    "b": {
      "type": "number",
      "description": "두 번째 숫자"
    }
  },
  "required": ["operation", "a", "b"]
}
```

```
"b": {
  "type": "number",
  "description": "두 번째 숫자"
},
"required": ["operation", "a", "b"]
}
```

≡ Function Calling

≡ 사칙 연산

Function Calling을 사용하여 사칙 연산을 수행하는 코드를 작성해 보세요.

math_main_console.py, math_functions.py

출력 결과

사용자 : 10과 3을 곱해줘

출력 메시지 :

```
ChatCompletionMessage(content=None, refusal=None, role='assistant', annotations=[], audio=None,
function_call=None, tool_calls=[ChatCompletionMessageFunctionToolCall(id='call_3oWwcSn61XwSM9Uly8hN72Yb',
function=Function(arguments={'operation': 'mul', 'a': 10, 'b': 3}, name='calculate'), type='function')])
```

AI : 10과 3을 곱하면 결과는 30입니다. 추가로 계산이 필요하시면 말씀해 주세요!

사용자 : exit

Function Calling

테이블 생성

Function Calling을 사용하여 데이터 베이스에 대한 CRUD 작업을 수행하는 코드를 작성해 보세요.

db_init.py

요구 사항

다음과 같은 코드를 사용하여 데이터 베이스에 샘플용 데이터를 추가합니다.

```
CREATE TABLE IF NOT EXISTS products(  
    product_id INTEGER PRIMARY KEY,  
    name TEXT NOT NULL,  
    price INTEGER NOT NULL,  
    stock INTEGER NOT NULL  
)
```

```
products = [  
    (1, 'Keyboard', 30000, 15),  
    (2, 'Mouse', 15000, 40),  
    (3, 'Monitor', 200000, 7),  
    (4, 'USB Cable', 5000, 100),  
]
```

테이블(T): products

	product_id	name	price	stock
	필터	필터	필터	필터
1	1	Keyboard	30000	15
2	2	Mouse	15000	40
3	3	Monitor	200000	7
4	4	USB Cable	5000	100

Function Calling

테이블 생성

Function Calling을 사용하여 데이터 베이스에 대한 CRUD 작업을 수행하는 코드를 작성해 보세요.

db_main.py, db_functions.py

출력 결과

사용자 : 상품 1의 이름과 단가를 알려줘

DEBUG: ...중략

```
function=Function(arguments={"product_id": 1, "field": "name"}, name='get_product_info'), type='function'),
ChatCompletionMessageFunctionToolCall(id='call_9STwewlegK1Hb5hnuZh6qV9g', function=Function(arguments={"product_id": 1, "field":
"price"}, name='get_product_info'), type='function'))]
```

AI : 상품 1의 이름은 "Keyboard"이며, 단가는 30,000원입니다.

사용자 : 10만원짜리 하드 디스크 HDD를 10개 추가하는 코드 작성

DEBUG: ...중략

```
function=Function(arguments={"name":"Hard Disk Drive(HDD)","price":100000,"stock":10}, name='add_product'), type='function'))]
```

AI : 새로운 상품 "Hard Disk Drive(HDD)"가 단가 100,000원으로 재고 10개가 추가되었습니다.

사용자 : exit

	product_id	name	price	stock
	필터	필터	필터	필터
1	1	Keyboard	30000	15
2	2	Mouse	15000	40
3	3	Monitor	200000	7
4	4	USB Cable	5000	100
5	5	Hard Disk Drive (HDD)	100000	10

Function Calling

사칙 연산

streamlit과 Function Calling을 사용하여 사칙 연산을 수행하는 코드를 작성해 보세요.

math_main_streamlit.py, math_functions.py

사용 예시



계산기 챗봇 (Function Calling)



10과 3을 곱해줘



10과 3을 곱하면 결과는 30입니다.



30 나누기 6은?



30을 6으로 나누면 결과는 5입니다.

계산을 입력하세요 (exit 입력 시 종료)



출력 결과

DEBUG :

```
ChatCompletionMessage(content=None, refusal=None, role='assistant',
annotations=[], audio=None, function_call=None,
tool_calls=[ChatCompletionMessageFunctionToolCall(id='call_SkxTDpjXibz
26aNBfzElfQ8z',
function=Function(arguments={'operation':'mul','a':10,'b':3},
name='calculate'), type='function')])
```

DEBUG :

```
ChatCompletionMessage(content=None, refusal=None, role='assistant',
annotations=[], audio=None, function_call=None,
tool_calls=[ChatCompletionMessageFunctionToolCall(id='call_MrcA807ZX
hnJ8PiGnEvwbErn',
function=Function(arguments={'operation':'div','a':30,'b':6},
name='calculate'), type='function')])
```

AI Agent

LangChain

 python

 NumPy

 pandas

 matplotlib

Cafe LangChain

카페에서 발생할 수 있는 주문을 랭체인을 사용하여 코드를 작성해 보세요.

langchain_barista_at_cafe.py

질문 내용

system : 너는 커피 전문점의 바리스타야. 항상 친절하고 메뉴를 추천해주는 말투로 답변해.
user : 오늘 너무 피곤한데 어떤 커피가 좋을까?

출력 결과

response

```
content='오늘 피곤하신가 보군요! 그런 날엔 진한 에스프레소나 카페 아메리카노를 추천드려요. 에스프레소는 강한 커피의 맛과 향을 즐길 수 있고, 아메리카노는 물을 더해 부드럽게 즐길 수 있습니다. 만약 좀 더 달콤한 것이 필요하시다면 카라멜 마키아토도 기분을 좋' additional_kwargs={'refusal': None}
response_metadata={'token_usage': {'completion_tokens': 100, 'prompt_tokens': 53, 'total_tokens': 153, 'completion_tokens_details': {'accepted_prediction_tokens': 0, 'audio_tokens': 0, 'reasoning_tokens': 0, 'rejected_prediction_tokens': 0}, 'prompt_tokens_details': {'audio_tokens': 0, 'cached_tokens': 0}}, 'model_provider': 'openai', 'model_name': 'gpt-4o-2024-08-06', 'system_fingerprint': 'fp_ba157e313e', 'id': 'chatcmpl-Dk41GtVaZhI4qRMtaVZfCzJPNuTwB', 'service_tier': 'default', 'finish_reason': 'length', 'logprobs': None} id='lc_run--019e6892-c4ea-72b0-bbb4-a06705d6aa03-0' tool_calls=[] invalid_tool_calls=[] usage_metadata={'input_tokens': 53, 'output_tokens': 100, 'total_tokens': 153, 'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 0, 'reasoning': 0}}
```

response.content

오늘 피곤하신가 보군요! 그런 날엔 진한 에스프레소나 카페 아메리카노를 추천드려요. 에스프레소는 강한 커피의 맛과 향을 즐길 수 있고, 아메리카노는 물을 더해 부드럽게 즐길 수 있습니다. 만약 좀 더 달콤한 것이 필요하시다면 카라멜 마키아토도 기분을 좋

Conversation Memory

Conversation Memory 클래스를 사용하여 카페 상황을 랭체인으로 구현해 보세요.

memory_class_01.py

요구 사항

ChatOpenAI(model="gpt-5-nano") 모델 생성
ConversationBufferMemory를 사용하여 이전 대화 저장
메모리 변수 이름은 "history" 사용
return_messages=True 설정
ChatPromptTemplate 구성

- system 역할 지정
- MessagesPlaceholder로 이전 대화 삽입
- 사용자 입력 처리

prompt | llm 형태의 LCEL 체인 구성

연속 대화 처리

1. 커피 추천
2. 차가운 음료 변경
3. 어울리는 디저트 추천

대화 후 save_context()로 메모리 저장
마지막에 전체 대화 기록 출력

결과 참조 파일 : memory_class_01.py.result.txt

Conversation Memory

Conversation Memory 클래스를 사용하여 카페 상황을 랭체인으로 구현해 보세요.

memory_class_02.py

요구 사항

ChatOpenAI(model="gpt-5-nano") 모델 생성

ConversationBufferWindowMemory 사용

최근 대화 3개만 기억하도록 k=3 설정

메모리 변수 이름은 "history" 사용

return_messages=True 설정

ChatPromptTemplate 구성

- system 역할 지정
- MessagesPlaceholder 사용
- 사용자 입력 처리

prompt | llm 형태의 LCEL 체인 구성

연속 주문 대화 처리

- 음료 주문
- 사이즈 변경
- 얼음 양 조절
- 디저트 추가 주문
- 첫 주문 메뉴 질문

save_context()를 사용하여 대화 저장

최근 대화만 기억되는 결과 확인

결과 참조 파일 : memory_class_02.py.result.txt

Conversation Memory

Conversation Memory 클래스를 사용하여 카페 상황을 랭체인으로 구현해 보세요.

memory_class_03.py

요구 사항

ChatOpenAI(model="gpt-5-nano") 모델 생성

ConversationSummaryMemory 사용

이전 대화를 자동 요약하여 저장

return_messages=True 설정

ChatPromptTemplate 구성

- system 역할 지정
- MessagesPlaceholder 사용
- 사용자 입력 처리

prompt | llm 형태의 LCEL 체인 구성

연속 주문 대화 처리

- 음료 주문
- 샷 추가
- 디저트 추천
- 이전 주문 메뉴 확인
- 추가 디저트 추천

save_context()를 사용하여 대화 저장

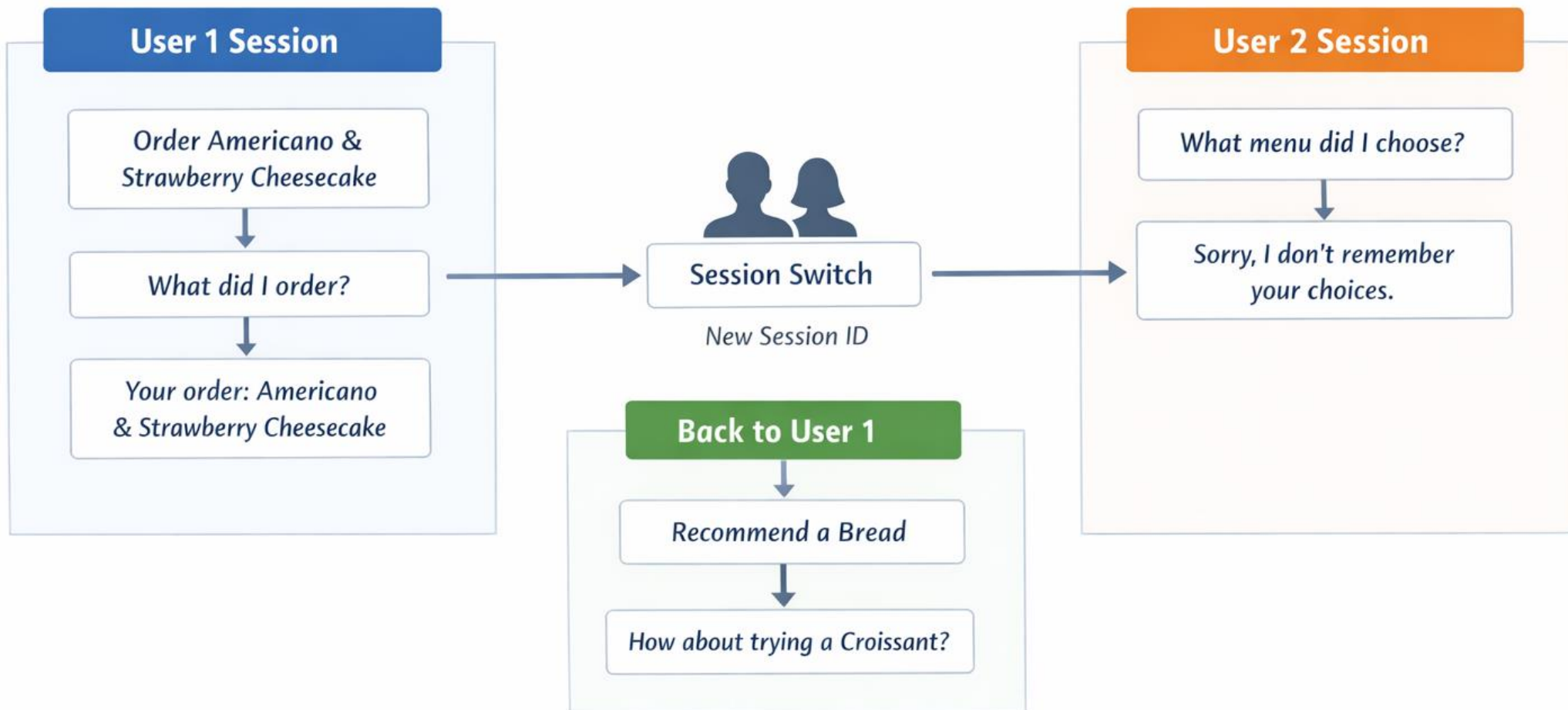
마지막에 memory.buffer로 요약 메모리 출력

결과 참조 파일 : memory_class_03.py.result.txt

Cafe LangChain

카페에서 발생할 수 있는 주문을 랭체인을 사용하여 코드를 작성해 보세요.

langchain_cafe_chatting.ipynb, langchain_cafe_chatting_result.txt



Cafe LangChain

카페에서 발생할 수 있는 주문을 랭체인을 사용하여 코드를 작성해 보세요.

langchain_cafe_chatting.ipynb, langchain_cafe_chatting_result.txt

구현 절차

1. model 생성
2. RunnableWithMessageHistory 객체 생성
 - ① 세션 관리를 위한 함수 정의
 - ② config(설정) 사전 정의
 - ③ invoke() 함수 호출
 - ④ 결과 출력 (content 속성)

Cafe LangChain

카페에서 발생할 수 있는 주문을 랭체인을 사용하여 코드를 작성해 보세요.

langchain_cafe_chatting.ipynb, langchain_cafe_chatting_result.txt

시스템 메시지(규칙)와 문의

```
SystemMessage(content="당신은 카페 직원이니, 친절하게 메뉴를 안내하  
세요.")
```

```
HumanMessage(content="빵/음료수/케이크 있나요?")],
```

안녕하세요! 저희 카페에 오신 것을 환영합니다. 저희 메뉴에는 다양한 빵,
음료수, 그리고 케이크가 준비되어 있습니다.

빵은 크루아상, 바게트, 팔빵, 음료수는 아메리카노, 라떼, 과일 주스 케이크
는 초콜릿 케이크, 딸기 치즈케이크, 당근 케이크 등이 있습니다.

원하시는 메뉴가 있으시면 말씀해 주세요! 또는 추천이 필요하시면 말씀해
주시면 도와드리겠습니다.

추천 메뉴 요청

```
HumanMessage(content="그럼 케이크와 음료를 추천해주세요.")
```

네, 좋은 선택이세요! 케이크와 음료를 조합해서 추천해 드릴게요.

케이크 추천:

- 딸기 치즈케이크: 신선한 딸기가 올라가 있어 상큼함과 부드러움이 잘 어
우러져 많은 분들이 좋아하는 케이크입니다.

음료 추천:

- 바닐라 라떼: 부드러운 우유와 바닐라 시럽이 어우러진 달콤한 맛의 음료
로, 딸기 치즈케이크와 환상적인 조화를 이룹니다.

Cafe LangChain

카페에서 발생할 수 있는 주문을 랭체인을 사용하여 코드를 작성해 보세요.

langchain_cafe_chatting.ipynb, langchain_cafe_chatting_result.txt

주문 하기

```
HumanMessage(content="아메리카노와 딸기 치즈케이크 주문할께요.")
```

감사합니다! 아메리카노와 딸기 치즈케이크 주문하신 것으로 확인해 드리겠습니다.

잠시만 기다려 주세요. 신선하게 준비해 드리겠습니다!

주문한 메뉴 다시 묻기

```
HumanMessage(content="제가 주문한 메뉴가 뭐였죠?")
```

주문하신 메뉴는 아메리카노와 딸기 치즈케이크입니다.

준비가 될 때까지 잠시 기다려 주세요!

다른 도움이 필요하시면 언제든지 말씀해 주세요.

Cafe LangChain

카페에서 발생할 수 있는 주문을 랭체인을 사용하여 코드를 작성해 보세요.

langchain_cafe_chatting.ipynb, langchain_cafe_chatting_result.txt

다른 손님(세션 변경)

```
config = {"configurable": {"session_id": "customer02"}}  
HumanMessage(content="제가 무슨 메뉴를 골랐죠?")
```

죄송하지만, 제가 지금까지의 대화를 기억하지 못해서 어떤 메뉴를 골랐는지 알 수 없습니다.

어떤 메뉴를 생각하고 계신지 말씀해 주시면 도와드릴 수 있습니다!

다시 첫 손님으로 복귀

```
HumanMessage(content="아까 추천해준 케이크 말고, 빵도 하나 추천해주세요.")
```

물론입니다! 빵 중에서 추천드릴 만한 것은 크루아상입니다.

크루아상은 바삭한 겉과 부드러운 속이 특징으로, 고소한 버터 맛이 일품입니다.

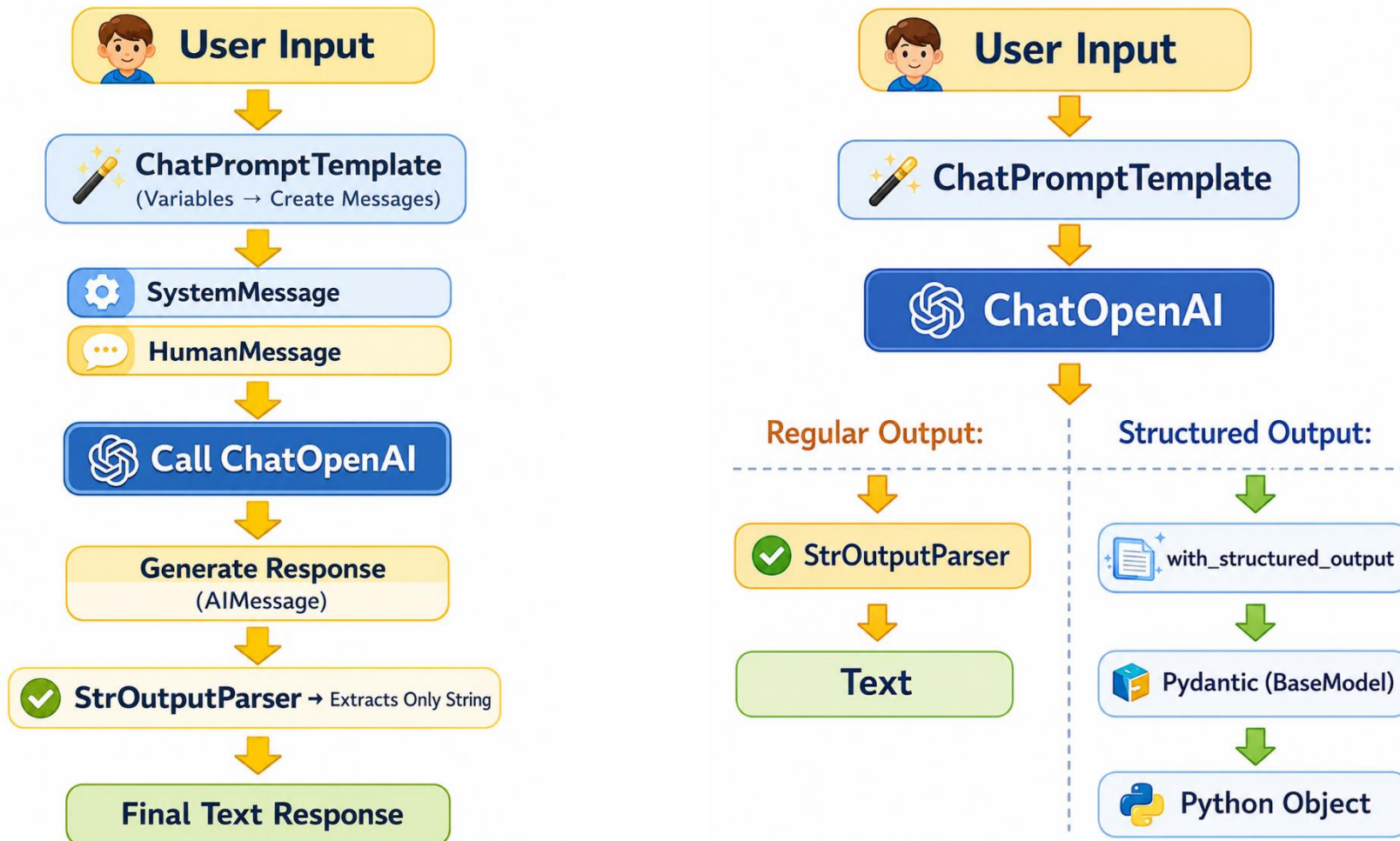
커피와 함께 즐기기에 정말 잘 어울립니다.

여기에 추가로 크루아상을 주문하실까요? 아니면 다른 종류의 빵을 원하시면 말씀해 주세요!

LCEL 사용해보기

LCEL을 사용하고 출력 형식을 '문자열 형식'과 'Pydantic 형식'으로 출력해 봅니다.

lcel_in_cafe.ipynb



LCEL 사용해보기

카페에서 발생할 수 있는 일을 LCEL을 사용하여 코드를 작성해 보세요.

lcel_in_cafe.ipynb

메시지 기반 호출(카페 역할극)

① 모델 생성

```
model = ChatOpenAI(model="gpt-4o-mini", temperature=0.3, max_tokens=100)
```

```
messages = [
```

```
    SystemMessage(content="너는 카페에서 일하는 직원이다. 손님에게 짧고 친절하게 응대해줘."),
```

```
    HumanMessage(content="안녕하세요. 케이크랑 음료 추천해 주세요."),
```

```
]
```

```
model.invoke(messages)
```

```
AIMessage(content='안녕하세요! 저희 인기 케이크는 초코 케이크와 레드벨벳 케이크입니다. 음료는 아메리카노와 바닐라 라떼가 잘 어울려요. 어떤 걸 드시고 싶으신가요?', additional_kwargs={'refusal': None}, response_metadata={'token_usage': {'completion_tokens': 60, 'prompt_tokens': 48,
```

```
... 중략
```

```
'service_tier': 'default', 'finish_reason': 'stop', 'logprobs': None}, id='lc_run--019bf8de-f5b5-7fb2-95fb-b70ef5f13aba-0', tool_calls=[], invalid_tool_calls=[], usage_metadata={'input_tokens': 48, 'output_tokens': 60, 'total_tokens': 108, 'input_token_details': {'audio': 0, 'cache_read': 0}, 'output_token_details': {'audio': 0, 'reasoning': 0}})
```

LCEL 사용해보기

카페에서 발생할 수 있는 일을 LCEL을 사용하여 코드를 작성해 보세요.

lcel_in_cafe.ipynb

OutputParser

```
from langchain_core.output_parsers import StrOutputParser
```

```
parser = StrOutputParser()
```

```
# 인보크한 모델을 출력 parser에게 다시 인보크시켜서 필요한 문자열만 출력해 줍니다.
```

```
result = model.invoke(messages)
```

```
parser.invoke(result)
```

```
'안녕하세요! 저희 카페의 초코 케이크와 아메리카노 조합이 정말 인기 있어요. 달콤한 케이크와 진한 커피가 잘 어울린답니다. 혹시 다른 취향이 있으신가요?'
```

LCEL 사용해보기

카페에서 발생할 수 있는 일을 LCEL을 사용하여 코드를 작성해 보세요.

lcel_in_cafe.ipynb

LCEL 체인

파이프(|) 연산자는 Runnable 합성 연산자라고 합니다.

왼쪽의 출력 정보를 오른쪽의 입력 정보로 입력합니다.

```
chain = model | parser
```

```
chain.invoke(messages)
```

‘안녕하세요! 저희 인기 케이크는 초코 케이크와 치즈 케이크입니다. 음료는 아메리카노나 바닐라 라떼가 잘 어울려요. 어떤 걸 드시겠어요?’

LCEL 사용해보기

카페에서 발생할 수 있는 일을 LCEL을 사용하여 코드를 작성해 보세요.

lcel_in_cafe.ipynb

프롬프트 템플릿

```
# ChatPromptTemplate로 카페 상황 일반화
from langchain_core.prompts import ChatPromptTemplate

system_template = "너는 {place}에서 일하는 {role}이다. 손님에게 짧고 친절하게 대답해줘."
human_template = "{menu} 추천해 주세요."

prompt_template = ChatPromptTemplate.from_messages([
    ("system", system_template),
    ("user", human_template),
])
```

LCEL 사용해보기

카페에서 발생할 수 있는 일을 LCEL을 사용하여 코드를 작성해 보세요.

lcel_in_cafe.ipynb

LCEL 체인 구성

```
chain = prompt_template | model | parser
```

```
chain.invoke({  
    "place": "카페",  
    "role": "직원",  
    "menu": "케이크랑 음료",  
})
```

‘저희 인기 메뉴인 초코 케이크와 카라멜 라떼를 추천해 드려요! 정말 맛있답니다.’

LCEL 사용해보기

카페에서 발생할 수 있는 일을 LCEL을 사용하여 코드를 작성해 보세요.

lcel_in_cafe.ipynb

여러 입력을 한 번에 처리(batch)

```
inputs = [  
    {"place": "카페", "role": "직원", "menu": "음료"},  
    {"place": "카페", "role": "직원", "menu": "빵"},  
]
```

```
results = chain.batch(inputs)
```

```
for rst in results:  
    print(rst)
```

어떤 종류의 음료를 원하시나요? 커피, 차, 또는 스무디 중에서 선택해 드릴 수 있어요!

저희의 크루아상과 바게트가 특히 인기 있어요! 둘 다 맛있으니 한번 드셔보세요.

LCEL 사용해보기

카페에서 발생할 수 있는 일을 LCEL을 사용하여 코드를 작성해 보세요.

lcel_in_cafe.ipynb

파이단틱 라이브러리

```
from typing import Literal
from pydantic import BaseModel, Field

# 반드시 { answer: 문자열, emotion: '친절' 또는 '중립' }의 구조로 대답해 줘야되.
class CafeResponse(BaseModel):
    answer: str = Field(description="카페 직원의 응답(짧게)")
    emotion: Literal["친절", "중립"] = Field(description="응답의 분위기")

structured_llm = model.with_structured_output(CafeResponse)
structured_chain = prompt_template | structured_llm
structured_chain.invoke({
    "place": "카페", "role": "직원", "menu": "초코 케이크랑 커피",
})

CafeResponse(answer='초코 케이크와 잘 어울리는 커피는 아메리카노나 카푸치노를 추천드려요! 달콤한 초코와 씹싸름한 커피의 조화가 정말 맛있습니다.',
emotion='중립')
```


스트림릿에 구현하기

카페에서 발생할 수 있는 주문을 streamlit을 사용하여 코드를 작성해 보세요.

langchain_cafe_chatting_streamlit.py

Cafe Chatbot

S 너는 빵, 음료, 케이크를 판매하는 카페 직원이야. 손님에게 항상 친절하고 간단하게 메뉴를 안내하고 추천해줘.

안녕하세요. 여기 케이크 있나요?

안녕하세요! 케이크에 대해 궁금한 점이 있으신가요? 제가 도와드릴 수 있는 부분이 있다면 말씀해 주세요!

그럼 케이크랑 잘 어울리는 음료 추천해 주세요.

케이크와 잘 어울리는 음료는 여러 가지가 있습니다. 몇 가지 추천해 드릴게요:

1. **커피**: 진한 에스프레소나 부드러운 카푸치노는 케이크와 잘 어울립니다. 특히 초콜릿 케이크와 함께하면 좋습니다.
2. **차**: 홍차나 녹차는 케이크의 달콤함을 잘 중화시켜 줍니다. 특히 레몬 케이크와 함께하면 상큼한 조화를 이룹니다.

아까 추천해 준 커피가 뭐였죠?

추천해 드린 커피는 **진한 에스프레소**와 **부드러운 카푸치노**입니다. 에스프레소는 강렬한 맛으로 초콜릿 케이크와 잘 어울리고, 카푸치노는 부드러운 거품이 케이크의 질감과 잘 어울립니다. 어떤 케이크와 함께 드실지에 따라 선택하시면 좋을 것 같습니다!

메뉴를 물어보세요 😊



AI Agent

RAG에 기반한 Chatbot 만들기



python



NumPy



pandas

matplotlib

☰ RAG에 기반한 Chatbot 만들기

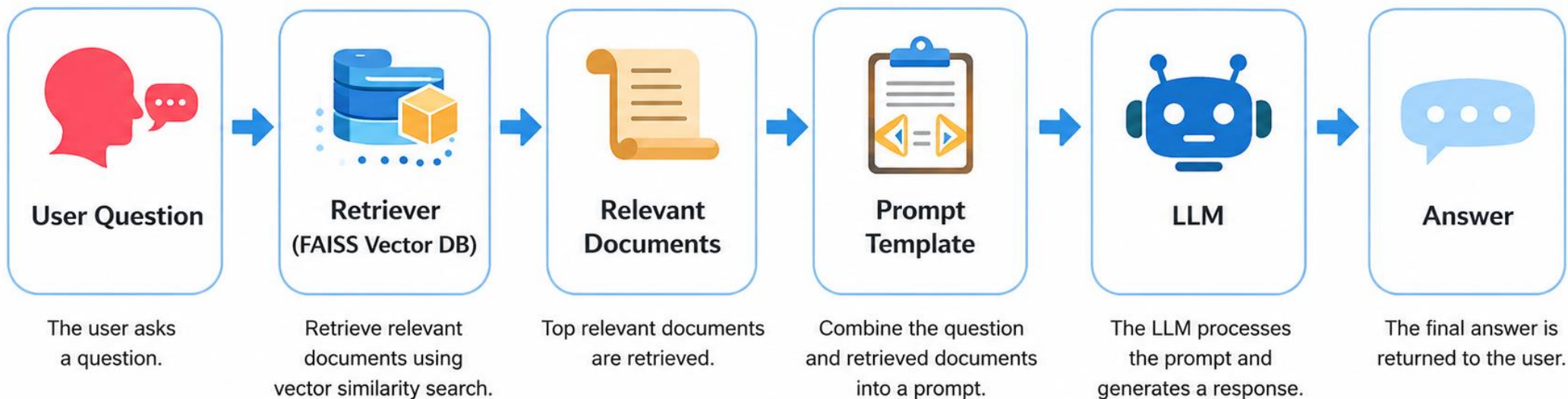
☰ Cafe RAG

카페에서 발생할 수 있는 주문 상황을 RAG를 사용하여 코드를 작성해 보세요.

rag_example.py, rag_example_결과.txt

흐름도 개요

Overview of Flow



≡ RAG에 기반한 Chatbot 만들기

≡ Cafe RAG

카페에서 발생할 수 있는 주문 상황을 RAG를 사용하여 코드를 작성해 보세요.

rag_example.py, rag_example_결과.txt

실습 순서

① 지식 문서 준비/Documents)

- 카페 메뉴 및 설명을 문장 형태의 텍스트로 준비
- → LLM이 직접 학습하지 않고 참고할 지식

② 문서 임베딩 생성(Embeddings)

- 각 문서를 의미 기반 숫자 벡터로 변환
- OpenAI Embedding API 사용(문서 등록 시 1회 비용 발생)

③ Vector DB 생성(FAISS)

- 임베딩된 문서를 FAISS 벡터 DB에 저장
- 의미적으로 유사한 문서를 빠르게 검색 가능

④ Retriever 구성

- Vector DB를 Retriever(검색기) 형태로 변환
- 질문이 들어오면 관련 문서 자동 검색

⑤ Prompt Template 설계

- 역할(Role): 카페 직원
- 입력 요소:
 - {context} → 검색된 문서
 - {question} → 사용자 질문
- 규칙: 정보에 없는 내용은 답변하지 않음

⑥ RAG 체인 구성(LCEL)

- Retriever → Prompt → LLM → OutputParser
- 하나의 실행 파이프라인으로 연결

⑦ 질문 실행 및 답변 생성

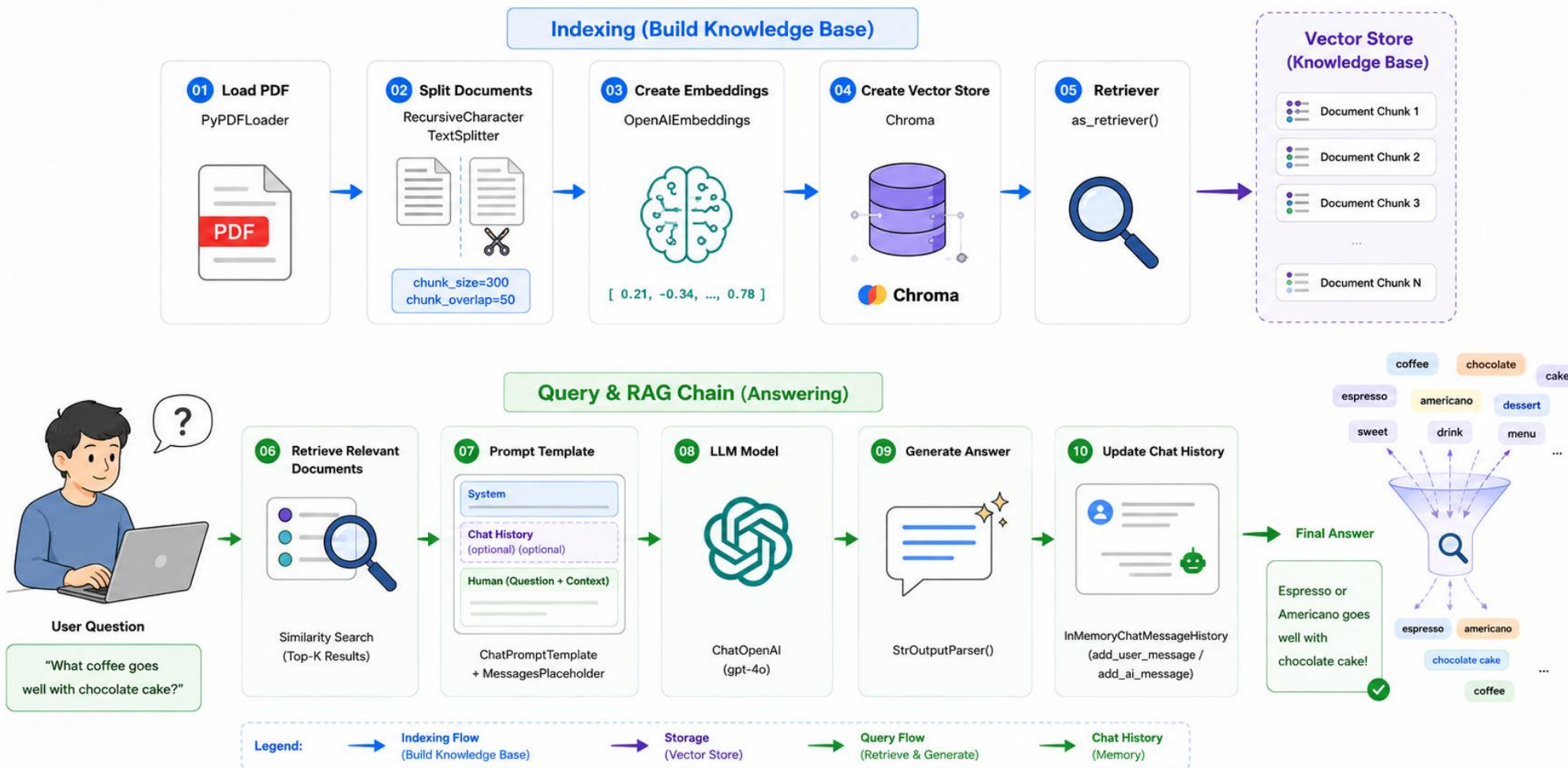
- 사용자 질문 입력
- 관련 문서만 근거로 답변 생성
- "아는 척하는 답변" 방지

RAG에 기반한 Chatbot 만들기

PDF 기반 RAG 챗봇 구현

카페 메뉴에 대한 정보를 pdf 파일과 RAG를 사용한 프로그램을 작성해 보세요.

rag_from_pdf.py, rag_from_pdf.py.result.txt



☰ RAG에 기반한 Chatbot 만들기

☰ PDF 기반 RAG 챗봇 구현

카페 메뉴에 대한 정보를 pdf 파일과 RAG를 사용한 프로그램을 작성해 보세요.

rag_from_pdf.py, rag_from_pdf.py.result.txt

요구 사항

PyPDFLoader를 사용하여 PDF 문서 로드
RecursiveCharacterTextSplitter로 문서 분할

- chunk_size=300
- chunk_overlap=50

OpenAIEmbeddings를 사용하여 임베딩 생성
Chroma Vector DB 생성 및 Retriever 구성
ChatPromptTemplate 구성

- system 역할 지정
- MessagesPlaceholder로 대화 기록 삽입
- context와 question 포함

ChatOpenAI(model="gpt-4o") 모델 생성
InMemoryChatMessageHistory로 대화 기록 관리

LCEL 기반 RAG Chain 구성
Retriever 검색 결과 출력
사용자 질문에 대한 최종 답변 출력
사용자 질문/AI 응답을 메모리에 저장
저장된 대화 기록 출력
데이터가 많은 경우 앞 10개까지만 출력

☰ RAG에 기반한 Chatbot 만들기

☰ Vector DB 활용

카페 정보 PDF 파일을 Vector DB에 저장하고 이를 사용하여 RAG를 구현한 프로그램을 작성해 보세요.

rag_01_create_vector_db.py

파일 내용

파일	설명
커피 매장 메뉴 및 이용 정보.pdf	RAG에 사용할 Pdf 파일 원본입니다.
rag_01_create_vector_db.py	PDF 파일을 읽어서 Vector DB에 저장합니다.
rag_02_read_vector_db.py	Vector DB에 저장된 정보를 출력해 봅니다. python -X utf8 rag_02_read_vector_db.py > rag_02_read_vector_db.py.result.txt
rag_03_chatbot_rag.py	해당 Vector DB를 사용하여 연속적인 대화를 생성해 봅니다.
rag_04_01_chatbot_streamlit.py	rag_04_01_chatbot_streamlit(최신 질문 내용만 보여 주기).png
rag_04_02_chatbot_streamlit.py	rag_04_02_chatbot_streamlit(질문과 답변만 보여 주기).png
rag_97.질문 예시.txt	여러 가지 상황에 대한 질문 예시 파일입니다.
rag_98.답변 예시.txt	특정 질문에 대한 답변 예시입니다.

≡ RAG에 기반한 Chatbot 만들기

≡ Vector DB 활용

카페 정보 PDF 파일을 Vector DB에 저장하고 이를 사용하여 RAG를 구현한 프로그램을 작성해 보세요.

rag_01_create_vector_db.py

최신 질문 내용만 보여 주기

☹ 카페 메뉴 RAG 챗봇

🗉 내가 지금까지 선택한 메뉴들을 정리해줘.

📄 검색된 문서

▼ 문서 1

★ 유사도 점수: 0.3644

선택되는 메뉴입니다. 헤이즐넛 라떼는 고소한 헤이즐넛 향이 특징인 음료입니다. 달콤하면서도 견과류 특유의 향이 느껴지며, 쿠키류 및 초코 디저트와 좋은 조화를 이룹니다. 달콤한 커피를 좋아하는 고객이 자주 선택합니다. 딸기 치즈케이크는 부드러운 식감과 상큼한 딸기 풍미가 특징인 디저트입니다. 치즈의 고소함과 딸기의 산뜻함이 자연스럽게 어우러지며, 라떼나 아메리카노와 함께 즐기기에 적합합니다. 부드러운 커피 음료와 함께할 경우 디저트의 풍미가 더욱 풍부하게 느껴집니다.

> 문서 2

> 문서 3

🗉 AI 답변

지금까지 선택한 메뉴들은 다음과 같습니다:

1. 초코 케이크와 아메리카노
2. 크루아상과 아메리카노

질문을 입력하세요



질문과 답변만 보여 주기

☹ 카페 메뉴 RAG 챗봇

🗉 초코 케이크와 잘 어울리는 커피 추천해줘.

🗉 초코 케이크와 잘 어울리는 커피로는 아메리카노와 콜드브루를 추천합니다. 초코 케이크의 진한 초콜릿 풍미와 씹싸름한 커피 음료가 함께 마실 경우 맛의 균형이 잘 맞아 더욱 풍부한 맛을 즐길 수 있습니다.

🗉 그 커피의 특징은 뭐야?

🗉 제공된 정보에 따르면, 초코 케이크와 잘 어울리는 커피로는 아메리카노와 콜드브루가 있습니다. 이 커피들은 씹싸름한 맛이 특징이며, 초코 케이크의 진한 초콜릿 풍미와 함께 마실 경우 맛의 균형이 잘 맞습니다. 아메리카노는 진한 커피의 풍미를 좋아하는 고객에게 적합하고,

🗉 비슷한 느낌의 다른 디저트도 추천해줘.

🗉 초코 케이크와 비슷한 느낌의 다른 디저트로 티라미수를 추천합니다. 티라미수는 커피 향과 크림의 부드러운 식감이 특징인 디저트로, 에스프레소 풍미가 은은하게 느껴집니다. 초코 케이크처럼 진한 디저트를 좋아하는 고객에게 인기가 많으며, 아메리카노나 콜드브루와 함께 즐기

🗉 너무 달지 않은 조합으로 바꿔줘

질문을 입력하세요



AI Agent

인터넷 검색을 활용한 챗봇 생성



python



NumPy



pandas

matplotlib

≡ 인터넷 검색을 활용한 챗봇 생성

≡ 핫한 질문

인터넷 검색을 활용하기 전에 우선 최근 hot issue가 무엇인지 찾아보도록 합니다.

핫한 질문 내용.txt

질문 사항

요즘 가장 핫한 내용을 duckduckgo-search를 사용해 질문에 불건데.. 뭐가 있을 지 질문 내용을 한글로 좀 만들어줘

답변 사항

🔥 한국 관련 핫이슈 (region=kr-ko 추천)

최근 한국에서 가장 화제가 되고 있는 IT · 기술 이슈는 무엇인가요?

2025년 한국 정부의 디지털 · AI 정책 주요 내용은 무엇인가요?

요즘 한국 스타트업 업계에서 주목받는 트렌드는 무엇인가요?

최근 한국 증시에서 가장 이슈가 된 산업은 무엇인가요?

최근 한국 소비 트렌드는 어떻게 변화하고 있나요?

🔥 사회 / 라이프스타일 트렌드

요즘 전 세계적으로 유행하고 있는 라이프스타일 트렌드는 무엇인가요?

최근 MZ세대의 주요 소비 트렌드는 무엇인가요? <- 예시에서 사용함

최근 주목받는 친환경 · 지속가능 트렌드는 무엇인가요?

요즘 가장 인기 있는 원격근무 · 업무 방식 트렌드는 무엇인가요?

최근 교육 분야에서 가장 큰 변화는 무엇인가요?

인터넷 검색을 활용한 챗봇 생성

덕덕고를 사용한 검색/답변

DuckDuckGo 검색 엔진과 LangChain을 활용하여 최신 뉴스 및 웹 문서를 검색합니다. `duckduckgo_search.ipynb`, `duckduckgo_search_result.txt`

요구 사항

1. OpenAI 연동

- .env 파일에서 OpenAI API Key를 로드할 것
- ChatOpenAI 모델을 사용하여 질문에 대한 응답을 생성할 것
- gpt-4o-mini 모델을 사용할 것

2. 웹 검색 기능

- DuckDuckGoSearchResults를 사용하여 검색 기능을 구현할 것
- 뉴스(news) 검색 기능을 지원할 것
- 한국 지역(kr-kr) 및 최근 일주일(time="w") 기준 검색을 수행할 것

3. 검색 결과 처리

- 검색 결과에서 URL 링크를 추출할 것
- URL 목록(link_list)을 생성하여 저장할 것

4. 웹 페이지 크롤링

- WebBaseLoader를 사용하여 웹 페이지 내용을 로드할 것
- alazy_load()를 이용한 비동기 방식으로 데이터를 수집할 것
- BeautifulSoup을 활용하여 HTML 문서를 파싱할 것
- 기사(article) 본문만 추출하도록 구현할 것

5. 문서 데이터 처리

- 기사 내용을 리스트(articles)에 저장할 것
- None 데이터 제거 후 문자열로 결합할 것
- LangChain Document 객체의 page_content를 활용할 것

6. 대화 메모리 관리

- InMemoryChatMessageHistory를 사용하여 대화 이력을 저장할 것
- 사용자 질문과 AI 응답을 메모리에 추가할 것

☰ 인터넷 검색을 활용한 챗봇 생성

☰ 덕덕고를 사용한 검색/답변

DuckDuckGo 검색 엔진과 LangChain을 활용하여 최신 뉴스 및 웹 문서를 검색합니다. `duckduckgo_search.ipynb`, `duckduckgo_search_result.txt`

요구 사항

7. Prompt 및 Chain 구성

ChatPromptTemplate을 사용하여 Prompt를 구성할 것
검색된 Context 기반으로 AI 답변을 생성할 것
Prompt에 MessagesPlaceholder를 적용할 것

8. 결과 출력 기능

사용자 질문 출력
AI 생성 답변 출력
토큰 사용량 출력

- 입력 토큰
- 출력 토큰
- 전체 토큰

☰ 인터넷 검색을 활용한 챗봇 생성

☰ 유튜브 영상 요약 및 키워드 분석 시스템

유튜브 영상을 요약하여 키워드들을 분석해주는 시스템을 구현해 보세요.

youtube_summary.ipynb, 동영상 요약 프롬프트 작성 예시.txt

요구 사항

YouTube 검색 API 기반 영상 검색 기능
영상 제목, 조회수, 업로드일, 재생시간 정보 출력
YouTube 자막(Transcript) 자동 수집
LangChain + OpenAI 기반 영상 내용 요약
긍정/부정 분위기 분석
주요 키워드 추출
60분 이하 영상 필터링 기능
형태소 분석(Okt)을 이용한 명사 추출
불용어 제거 및 단어 빈도수 분석
matplotlib 기반 키워드 TOP10 시각화

인터넷 검색을 활용한 챗봇 생성

챗봇과 웹 검색 도구

인터넷 검색 기능을 활용하는 챗봇을 만들어 보세요.

streamlit_with_web_search.py

검색 결과

----- WEB SEARCH -----

인터넷 검색어 : 허수아비 드라마

검색 기간 : m

content="snippet: 2 days ago - 《허수아비》는 2026년 4월 20일부터 2026년 5월 26일까지 방송된 ENA 월화 드라마다. 연쇄살인사건의 진범을 수사하던 형사가 자신이 혐오하던 놈과 뜻밖의 공조 관계를 맺으면서 펼쳐지는 혐관 공조 수사극 · 이지현 : SBS 금토 ..., title: 허수아비 (드라마) - 위키백과, 우리 모두의 백과사전, link: [https://ko.wikipedia.org/wiki/허수아비_\(드라마\)](https://ko.wikipedia.org/wiki/허수아비_(드라마));nsnippet: 1 week ago - 2026년 4월 20일부터 2026년 5월 26일까지 〈클라이맥스〉의 후속으로 방영된 ENA 월화 드라마. 이준재 연쇄살인 사건을 모티브로 한 작품이다., title: 허수아비(드라마) - 나무위키, link: [https://namu.wiki/w/허수아비\(드라마\)](https://namu.wiki/w/허수아비(드라마));nsnippet: 1 week ago - 그러나 민지 역시 친구 정린이 죽었던 굴다리에서 '허수아비'에게 납치 및 강간 당하고, 인근 야산에서 강성 연쇄살인사건의 6번째 희생자가 된다. 시신의 음부에 카라멜이 박힌 채 발견된다., ... 이하 중략

GPT-4o Langchain Chat

S 너는 사용자를 돕기 위해 최선을 다하는 인공 지능 로봇이야.

How can I help you?

최근에 종료한 '허수아비' 드라마에 대하여 어떻게 생각해?

드라마 '허수아비'는 2026년 4월 20일부터 5월 26일까지 방영된 ENA의 월화 드라마로, 이준재 연쇄살인 사건을 모티브로 한 작품입니다. 주요 내용은 연쇄살인사건을 수사하던 형사가 자신이 혐오하던 범 죄자와 뜻밖의 공조 관계를 맺으며 전개되는 혐관 공조 수사극입니다.

이 드라마는 복잡한 사건의 본질을 파고드는 주인공의 모습을 통해 인간의 심리와 갈등을 깊이 있게 다루고 있으며, 과거의 죄책감을 안고 살아가는 캐릭터의 내면적 갈등도 중요한 테마로 자리잡고 있습니다. 최근에 종료된 만큼, 시청자들에게 대한 반응과 평가가 집중되고 있을 것으로 보입니다.

더 궁금한 점이 있으면 말씀해 주세요!

Your message



☰ 인터넷 검색을 활용한 챗봇 생성

☰ 챗봇과 유튜브 검색 도구

유튜브 검색 도구 기능을 활용하는 챗봇을 만들어 보세요.

streamlit_with_web_search.py

LangChain ChatBot

S 너는 AI 검색 비서이다.

필요 시 다음 도구를 적극 활용하라:

- 현재 시간 조회
- 인터넷 검색
- 유튜브 검색 및 영상 요약

 무엇을 도와드릴까요?

 최근에 종료한 '허수아비' 드라마에 대하여 어떻게 생각해?

 드라마 '허수아비'에 대한 최근 리뷰와 의견은 다음과 같습니다:

1. **장르물에 대한 긍정적인 평가:** 많은 리뷰에서 '허수아비'는 잘 만들어진 장르물로 평가받고 있습니다. 특히, 드라마의 구성과 스토리 전개가 신선하다는 의견이 많습니다.
2. **진범이 드러난 후의 전개:** 드라마는 전통적인 범인을 쫓는 이야기가 아닌, 이미 진범이 밝혀진 후에 등장하는 인물들의 이야기를 중심으로 진행되어 독특한 시각을 제공합니다. 이 점이 흥미롭다는 평가도 존재합니다.
3. **편안한 시청 가능성:** 일부 시청자는 드라마가 너무 예상 가능하다고 느껴, 스킵하면서 보더라도 큰 상관이 없다고 언급했습니다. 이는 드라마의 긴장감을 다소 낮추는 요소로 작용할 수 있습니다.
4. **작품성 중시:** 드라마는 유명 배우 캐스팅보다는 작품성을 중시하며, 그로 인해 제작비를 절감하려는 노력을 하고 있다는 점도 언급되었습니다.

이러한 다양한 반응 속에서 '허수아비'는 시청자들에게 색다른 즐거움을 제공한 것으로 보입니다.

질문을 입력하세요.



AI Agent

Travel Agent



python



NumPy



pandas

matplotlib

Travel Agent

여행 에이전트

여행 에이전트에 세부적인 내역입니다.

파일 구성

```
|  
| main.py  
| state.py  
|  
|—agents  
| | communicator.py : 최종 결과를 사용자에게 전달하는 역할  
| | executor.py : 최종 보고서를 생성해주는 역할  
| | planner.py : 출력 계획을 수립하는 역할  
| | research.py : 항공권/호텔/날씨 정보를 조사하는 역할  
| | supervisor.py : 출장 준비 시스템의 AI 총 책임자  
|  
|—graph  
| | workflow.py
```

```
|  
|—models  
| | task.py  
| | trip_plan.py  
|  
|—tools  
| | travel_tools.py  
|  
|—utils  
| | file_utils.py : 파일 유틸리티  
|
```

작업(Task)

LLM이 수행해야 할 작업들을 하나씩 관리하는 "작업 관리자 객체"입니다.

\models\task.py

세부 내용

여행 계획을 세우는 과정에서 각 에이전트에게 어떤 일을 시킬지 기록하고, 완료 여부를 관리하는 역할을 합니다.

agent

- 실행할 에이전트 종류를 제한
- 다음 4가지 중 하나만 허용
- "planner", "research", "executor", "communicator"

done: bool = False

- 작업 완료 여부

description: str

- 작업 설명(슈퍼바이저가 생성하는 task 내용)

done_at: str = ""

- 작업 완료 시간(문자열로 저장)

to_dict() 함수

- Task 객체를 dict 형태로 변환(저장/직렬화용)

State

LangGraph의 각 노드가 공유하고 업데이트하는 데이터 저장소입니다.

state.py

세부 내용

모든 Node들이 공유하는 저장소입니다.

```
messages: List[AnyMessage]
    • # 대화 메시지 기록(user / AI / system 메시지 전체)

task_history: List[Task]
    • # 에이전트 실행 이력(planner, research 등 task 단위 기록)

trip_plan: dict
    • # 출장 계획 정보(목적지, 기간, 비행편, 호텔, 날씨 정보 등등)

research_result: dict
    • # 조사 결과(항공, 호텔, 날씨 정보)

final_report: str
    • # 최종 생성된 보고서
```

세부 내용

trip_plan은 다음과 같이 구성이 되어 있습니다.

```
{
    "destination": "", # 목적지
    "period": "", # 기간
    "need_flight": True, # 항공편 필요 여부
    "need_hotel": True, # 호텔편 필요 여부
    "need_weather": True, # 날씨 정보 필요 여부
    "summary": ""
}
```

research_result는 다음과 같이 구성이 되어 있습니다.

```
flight 사전 : {"route", "price"}

hotel 사전 : {"hotel", "price"}

weather 사전 : {"location", "weather", "temperature"}
```

Travel Agent

Tool(툴)

Agent가 사용하기 위하여 참조하는 외부 서비스 정보입니다.

\tools\travel_tools.py

세부 내용

예를 들면, **비행편 정보/호텔 검색, 날씨 정보** 확인하기 등등입니다.

다음 코드는 현재 임의로 하드 코딩이 되어 있습니다.

실전에서는 API를 사용하는 것이 좋을 듯 합니다.

```
def search_flight(destination):  
    return {  
        "route": f"김포 → {destination}",  
        "price": 89000  
    }  
  
def search_hotel(destination):  
    return {  
        "hotel": f"{destination} 비즈니스 호텔",  
        "price": 120000  
    }
```

```
def get_weather(destination):  
    return {  
        "location": destination,  
        "weather": "맑음",  
        "temperature": "27도"  
    }
```

관련 API

항공권 검색 API

Amadeus

Skyscanner

Aviationstack

호텔 정보

Booking.com API

Hotels.com API

Expedia API

Agoda API

기상 API

OpenWeatherMap API

WeatherAPI

Tomorrow.io

기상청 API

≡ Travel Agent

≡ 파일 유틸리티

state(대화 메시지, 작업 이력 등이 포함)를 JSON 형식으로 저장해주는 함수 모음입니다.

\utils\file_utils.py

save_state 함수

```
# data 폴더 생성  
# 대화 메시지 저장  
# 작업 이력 저장  
# 나머지 상태 정보 저장  
# JSON 파일로 저장
```

≡ Supervisor

출장 준비 AI 시스템의 Supervisor(관리자)를 의미합니다.

\agents\supervisor.py

세부 내용

출장 준비를 제공해주는 AI 시스템의 Supervisor(관리자) 클래스입니다.

supervisor(state: State, model) 함수:

- PromptTemplate 객체 생성
- chain 구성 = prompt | model
- task 객체 : chain.invoke() 호출 결과
- 에이전트 실행 이력에 추가("task_history")
- AIMessage 메시지 추가
- state 객체 반환

출장 계획 수립

출장에 필요한 정보(목적지, 기간 등)의 계획을 수립하는 주체입니다.

\agents\planner.py

세부 내용

출장 준비를 제공해주는 AI 시스템의 Supervisor(관리자) 클래스입니다.

planner_agent(state: State, model) 함수:

- PromptTemplate 객체 생성
- chain 구성 = prompt | model | StrOutputParser()
- result 객체 : chain.invoke() 호출 결과
- 여행 플랜(trip_plan) 객체를 json으로 읽음
- state에 여행 플랜 객체 저장
- 마지막 실행 이력("task_history") 정보 갱신
- state 객체 반환

Travel Agent

Research

항공권/호텔/날씨 정보 등을 조사하는 Agent입니다.

\agents\research.py

세부 내용

출장 준비를

research_agent(state: State) 함수:

- 목적지 정보 읽기 : state["trip_plan"]["destination"]
- search_flight 함수 사용 : 경로(route)와 단가(price) 정보 취득
- search_hotel 함수 사용 : 호텔(hotel)과 단가(price) 정보 취득
- get_weather 함수 사용 : 위치(location)와 날씨(weather), 온도(temperature) 정보 취득
- state에 조사 결과(항공, 호텔, 날씨 정보_research_result)를 저장
- state에 AIMessage 메시지 추가
- 마지막 실행 이력("task_history") 정보 갱신
- state 객체 반환

executor

최종 결과 사용자 전달

\agents\executor.py

세부 내용

최종적으로 생성할 보고서를 작성합니다.

executor_agent(state: State) 함수:

- state에서 'research_result' 정보 읽어 오기
- 전체 비용(total_cost) 구하기 : 비행기 가격(flight_price) + 호텔 가격(hotel_price)
- 보고서(report) 작성
 - 목적지, 기간, 항공권, 호텔, 날씨
 - 총 예상비용
 - 체크리스트
 - 노트북
 - 충전기
 - 명함
 - 출장비 카드
- state에 최종 생성 보고서('final_report') 저장
- 마지막 실행 이력("task_history") 정보 갱신
- state 객체 반환

≡ Travel Agent

≡ communicator

최종 보고서를 사용자 메시지로 저장하고, 마지막 작업을 완료 처리한 후 상태(State)를 반환하는 함수입니다. `\agents\communicator.py`

세부 내용

communicator(state: State) 함수:

- state에서 최종 보고서(final_report) 정보 읽어 오기
- state의 messages에서 최종 보고서를 AIMessage 메시지 형식으로 추가
- 마지막 실행 이력("task_history") 정보 갱신
- state 객체 반환

워크플로우

여행 계획에 사용할 워크플로우(Graph) 생성을 생성합니다.

\graph\workflow.py

세부 내용

여행 계획에 사용할 워크플로우(Graph) 생성을 생성합니다.

create_graph(model): 함수

- travel_graph 그래프 객체 생성
- 노드 등록 : supervisor, planner, research, executor, communicator
- 에지 생성 :
- 시작 노드 지정 : START→supervisor
- supervisor 분기
 - "planner" → Planner 실행
 - "research" → Research 실행
 - "executor" → Executor 실행
 - "communicator" → Communicator 실행

• 후속 흐름

- planner→research
- research→executor
- executor→communicator
- communicator→END
- 그래프 객체 반환
 - compile된 객체 travel_graph를 반환

≡ 핫한

인터넷

main.py

세부 내용

출장 준비를